

Développement front

MMI 2 – CM#5 S3



Danielo **JEAN-LOUIS**
Michele **LINARDI**

Réalisons des crêpes

- Listons l'ensemble des instructions (et besoins) permettant de réaliser des crêpes

Algorithme

- Suite d'instructions qui doivent mener au résultat attendu. Ex : recette de cuisine
- Les instructions doivent être compréhensibles par la personne qui va les exécuter
 - En programmation, il faut utiliser les bonnes instructions

Algorithme

- Est agnostique
 - Peut être écrit dans un langage de programmation ou en pseudo-code pour plus se concentrer sur la logique
- Constitué d'opérations de base : boucle, addition, condition...
- Peut appeler un autre algorithme et ainsi de suite

Algorithme

- Décompose le problème. Un peu comme en mathématiques quand on résout une équation
- A noter que plusieurs algorithmes peuvent mener au même résultat (pas forcément avec la même efficacité)

Algorithme des crêpes

- Ingrédients : 125 g de farine, 30 g de beurre fondu, 1 pincée de sel, 3 œufs, 30 cl de lait, 2 càs d'eau, 50 g de sucre
- Matériel : Fouet, saladier, balance, poêle, tamis

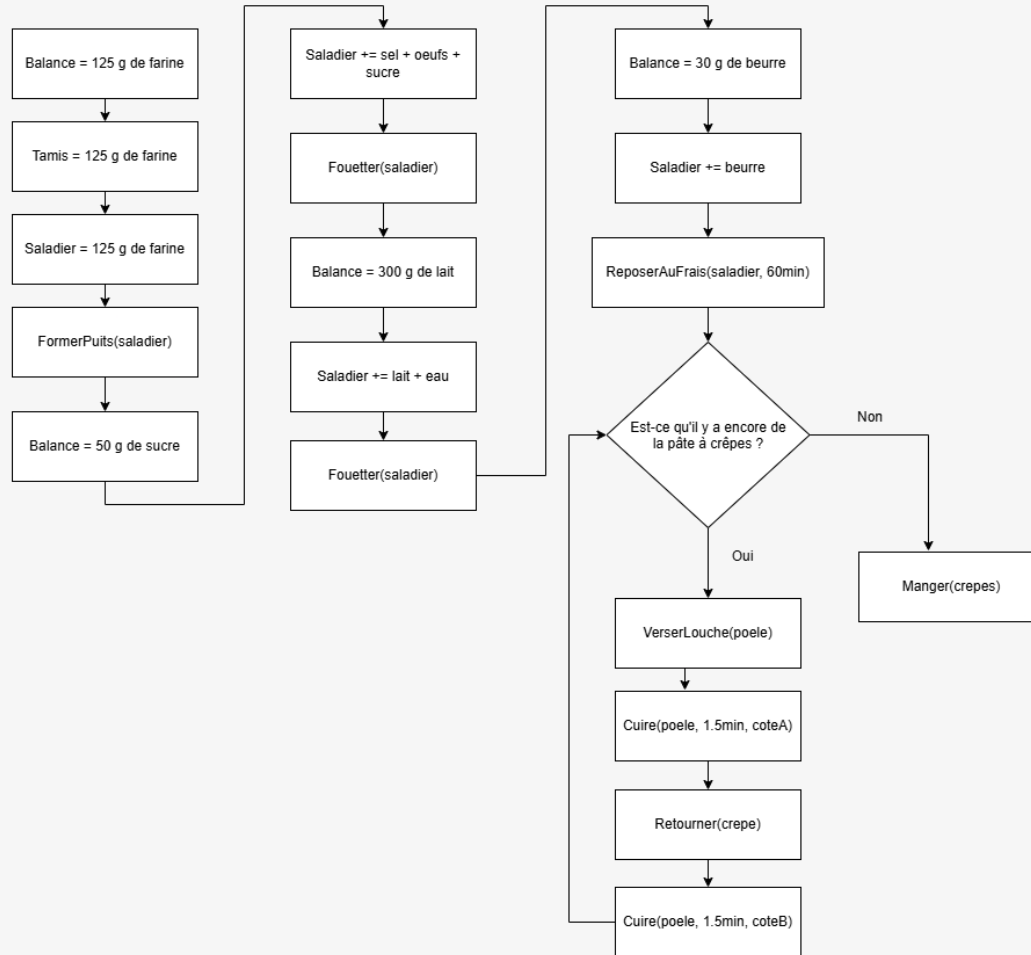
Algorithme des crêpes

- Recette :
 - Tamiser la farine
 - Former un puits avec la farine
 - Verser au centre le sel, les œufs et le sucre
 - Fouetter
 - Ajouter progressivement le lait puis l'eau

Algorithme des crêpes

- Recette (suite) :
 - Ajouter le beurre
 - Fouetter le tout
 - Reposer la pâte au frais une heure min
 - Verser une louche de pâte dans une pêle chaude
 - Cuire 2 minutes par côté jusqu'à épuisement de la pâte

Algorithme des crêpes (schéma)



Vous retrouverez le schéma en grand dans la ressource

Vocabulaire – Liste non exhaustive

- Entrée : Données de départ
- Sortie : Résultat final
- Variable : Stocke une valeur
- Instruction : Action à effectuer
- Type de sortie : Restreint la valeur que peut accepter une variable
 - Note : Cette notion n'existe pas en javascript

Source(s) :

- <https://perso.liris.cnrs.fr/pierre-antoine.champin/enseignement/algo/cours/algo/index.html>
- https://fr.wikipedia.org/wiki/Organigramme_de_programmation

Vocabulaire – Liste non exhaustive

- Condition : A effectuer si la condition est vraie
- Affectation : Donner une valeur à une variable
- Complexité : Nombre d'opérations à effectuer pour résoudre le problème

Source(s) :

- <https://perso.liris.cnrs.fr/pierre-antoine.champin/enseignement/algo/cours/algo/index.html>
- https://fr.wikipedia.org/wiki/Organigramme_de_programmation

Algorithme

- Éléments à identifier :
 - Quelles entrées sont attendues ?
 - Qu'est-ce qui va être retourné ?
 - Quels types vont être utilisés ?
 - **Non eligible en javascript**

Critères d'évaluation d'un algorithme

- Performance : Combien de temps met-il pour retourner le résultat ?
 - On peut également évaluer sa mise à l'échelle
- Résultat : Donne-t-il le résultat attendu ?
 - Penser à tester “tous” les cas

Premier algorithme : la multiplication

Compte-tenu de notre connaissance de l'addition. Essayons d'écrire l'algorithme de la multiplication de deux entiers.

Quel est le raisonnement ?

Premier algorithme : la multiplication

Compte-tenu de notre connaissance de l'addition. Essayons d'écrire l'algorithme de la multiplication de deux entiers.

$$2 \times 3 = 2 + 2 + 2$$

Une multiplication est la version compacte d'une addition

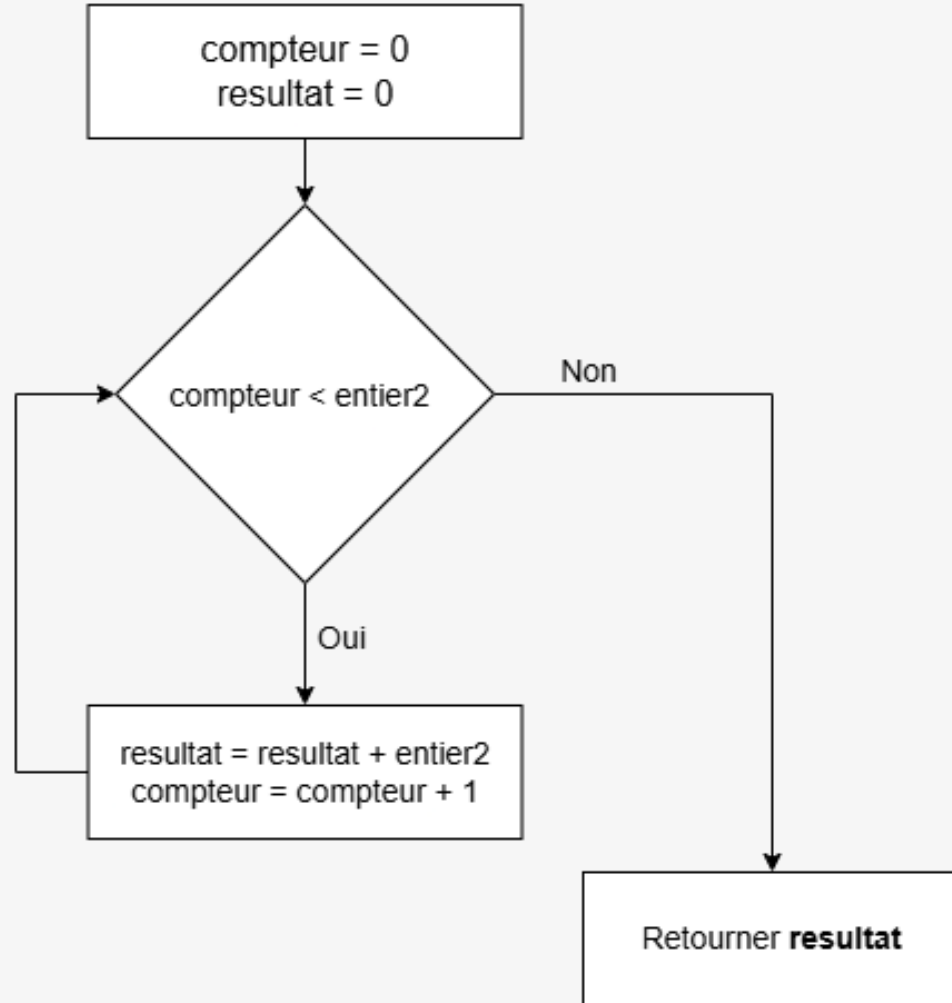
Premier algorithme : la multiplication

- Entrées : ?
- Sortie : ?
- Variables : ?

Premier algorithme : la multiplication

- Entrées : entier1 et entier2
- Sortie : resultat
- Variables : nombre d'addition, nombre à additionner, compteur d'itérations

Premier algorithme : la multiplication



Le tri

- Classique de l'algorithme
- Plusieurs algorithmes possibles
 - Du plus simple mais peu performant
 - Au plus complexe mais plus performant

Le tri

- Entrée : liste d'éléments à trier selon un prédicat
- Sortie : liste d'éléments triés

Tri à bulles

- Réorganise un tableau d'élément
- Algorithme de tri le plus simple mais le moins performant
 - Plus le tableau est grand, plus l'algorithme prendra du temps

Source(s) :

- https://fr.wikipedia.org/wiki/Tri_%C3%A0_bulles

Tri à bulles

- Fonctionnement :
 1. Parcours du tableau
 2. Comparaison de chaque élément avec le suivant
 3. Échanger l'élément actuel si celui suivant est plus petit
 4. Répéter sur l'ensemble du tableau
 5. Réitérer jusqu'à ce que le tableau soit trié

Source(s) :

- https://fr.wikipedia.org/wiki/Tri_%C3%A0_bulles

Tri à bulles



JS

```
const triABulles = (tableau) => {  
  for (let j = 0; j < tableau.length; j++) {  
    for (let i = 0; i < tableau.length; i++) {  
      if (tableau[i] > tableau[i + 1]) {  
        let temp = tableau[i];  
        tableau[i] = tableau[i + 1];  
        tableau[i + 1] = temp;  
      }  
    }  
  }  
  
  return tableau;  
};
```

Source(s) :

- https://fr.wikipedia.org/wiki/Tri_%C3%A0_bulles

Tri à bulles



JS

```
const triABulles = (tableau) => {  
  for (let j = 0; j < tableau.length; j++) {  
    for (let i = 0; i < tableau.length; i++) {  
      if (tableau[i] > tableau[i + 1]) {  
        let temp = tableau[i];  
        tableau[i] = tableau[i + 1];  
        tableau[i + 1] = temp;  
      }  
    }  
  }  
  
  return tableau;  
};
```

La première boucle parcourt le tableau. La seconde boucle compare chaque valeur avec la suivante et réordonne si besoin (instructions du if).

Source(s) :

- https://fr.wikipedia.org/wiki/Tri_%C3%A0_bulles

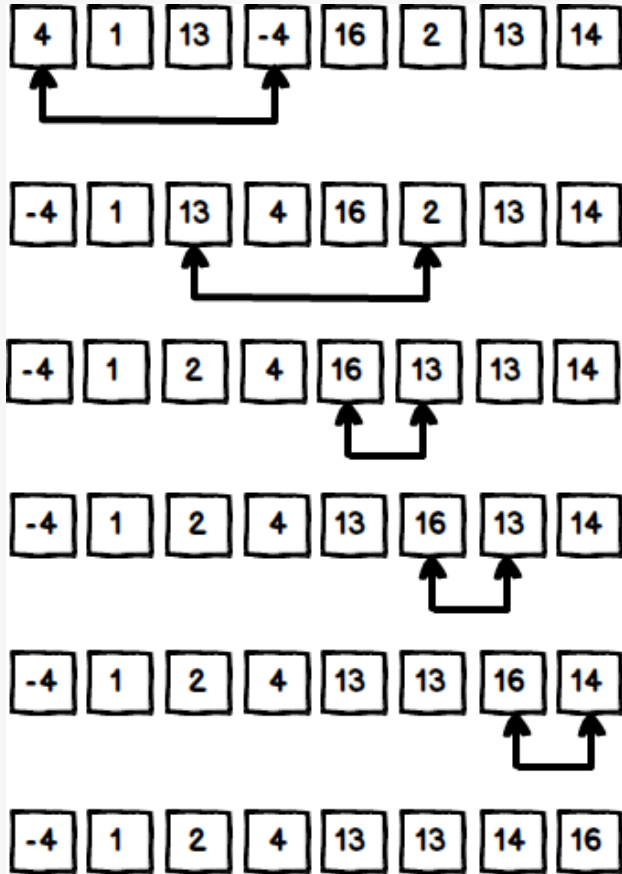
Tri par insertion

- Algorithme plus performant que le tri à bulles mais n'est pas le plus optimal pour autant
 - Plutôt performant sur les petites listes

Source(s) :

- <https://www.youtube.com/watch?v=Ns4TPTC8whw>

Tri par insertion



Fonctionnement :

- On cherche le plus petit élément du tableau et on l'échange avec l'élément qui est à sa place

Source : <https://waytolearnx.com/2019/09/tri-par-selection-en-javascript.html>

Source(s) :

- <https://www.youtube.com/watch?v=Ns4TPTC8whw>

Heureusement

Le tri natif

- La fonctionnalité est native en javascript grâce à la méthode **.sort()**
 - Elle trie le tableau **sur place**
 - Peut accepter en paramètre un prédicat
 - Tri par ordre croissant **naturel** par défaut



JS

```
const listePrenoms = ["Christophe", "Pauline", "Antoine", "Julie"];
listePrenoms.sort();
console.log(listePrenoms); // ["Antoine", "Christophe", "Julie", "Pauline"]
```

Source(s) :

- https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/sort

Le tri natif



```
const listeMois = ["Décembre", "Janvier", "Mai", "Février", "Novembre", "Août"];
listeMois.sort();
// ['Août', 'Décembre', 'Février', 'Janvier', 'Mai', 'Novembre']
console.log(listeMois);
```

Bien que listant les mois de l'année, la méthode `sort()` a trié par ordre alphabétique. Il va falloir passer par un tableau d'objets et un prédicat pour trier correctement les mois.

Source(s) :

- https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/sort

Le tri n

```
const listeMoisObjet = [
  {
    nom: "Décembre",
    nb: 12
  },
  {
    nom: "Janvier",
    nb: 1
  },
  {
    nom: "Mai",
    nb: 5
  },
  {
    nom: "Février",
    nb: 2
  },
  {
    nom: "Novembre",
    nb: 11
  },
  {
    nom: "Août",
    nb: 8
  }
]
```

```
listeMoisObjet.sort((a, b) => {
  return a.nb - b.nb
});
console.log(listeMoisObjet);
```

a.nb – b.nb trie les nombres par ordre croissant et l'inverse trie dans l'ordre décroissant

Source(s) :

- https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/sort

Le tri natif

- Tri lexicographique (du dictionnaire)



JS

```
const listeMois = ["Décembre", "Janvier", "Mai", "Février", "Novembre", "Août"];  
listeMois.sort((a, b) => a.localeCompare(b));  
console.log(listeMois);
```

Inverser “a” et “b” triera le tableau dans l’ordre décroissant

Source(s) :

- https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/sort

Autres méthodes natives pour les tableaux

- **.filter()** : Filtre en fonction d'un prédicat
- **.find()** : Retourne le premier élément trouvé
- **.every()** : Retourne "true" si **tous** les éléments du tableau vérifient un critère

Source(s) :

- https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/filter
- https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/find
- https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/every

Pratiquons ! – Appliquons les algorithmes

Pré-requis :

- Avoir la ressource `ressources/algorithmes/exercice`

A télécharger ici : https://github.com/DanYellow/cours/raw/refs/heads/main/s3-integration-web/cours-magistraux/numero-4/s3-integration-web_cours-magistraux_numero-4.ressources.zip

En résumé

Un algorithme est une suite d'instructions permettant de résoudre un problème.
Il contient des entrées et une sortie attendue.

Source(s) :

- https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/filter
- https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/find
- https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/every

Questions ?

